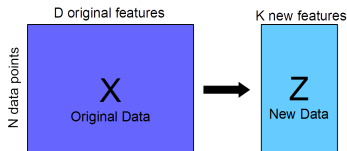


Dimensionality Reduction

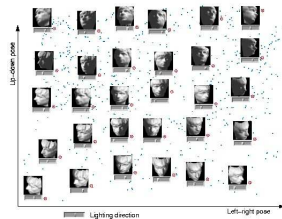
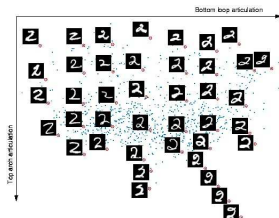
- Given: an $N \times D$ data matrix $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$, with $\mathbf{x}_n \in \mathbb{R}^D$
- Want a lower-dim. rep. as an $N \times K$ matrix $\mathbf{Z} = \{\mathbf{z}_1, \dots, \mathbf{z}_N\}$, $\mathbf{z}_n \in \mathbb{R}^K$
- $K \ll D \Rightarrow$ dimensionality reduction



- Learns a new feature representation of data with reduced dimensionality
- Don't want to lose much information about $\mathbf{X}$ while doing this: want to preserve the interesting/useful information in $\mathbf{X}$ and discard the rest
- Various ways to quantify what "useful" is (depends on what we want to learn)

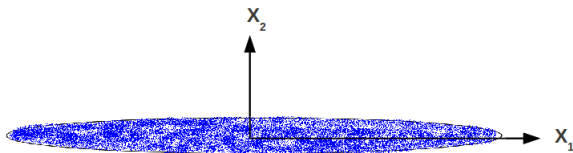
Why Dimensionality Reduction?

- To compress data by reducing dimensionality. E.g., representing each image in a large collection as a linear combination of a small set of “template” images
 - Also sometimes called **dictionary learning** (can also be used for other types of data, e.g., speech signals, text-documents, etc.)
- Visualization (e.g., by projecting high-dim data to 2D or 3D)



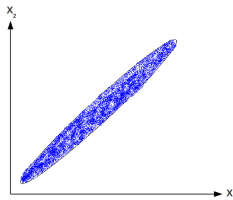
- To make learning algorithms run faster
- To reduce overfitting problem caused by high-dimensional data

Dimensionality Reduction: A Simple Illustration



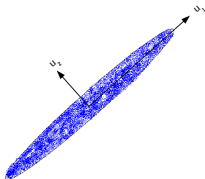
- Consider this 2 dimensional data
- Each example $\mathbf{x}$ has 2 features $\{x_1, x_2\}$
- Consider ignoring the feature x_2 for each example
- Each 2-dimensional example $\mathbf{x}$ now becomes 1-dimensional $\mathbf{x} = \{x_1\}$
- Are we losing much information by throwing away x_2 ?
- No. Most of the data spread is along x_1 (very **little variance** along x_2)

Dimensionality Reduction: A Simple Illustration



- Consider this 2 dimensional data
- Each example $\mathbf{x}$ has 2 features $\{x_1, x_2\}$
- Consider ignoring the feature x_2 for each example
- Each 2-dimensional example $\mathbf{x}$ now becomes 1-dimensional $\mathbf{x} = \{x_1\}$
- Are we losing much information by throwing away x_2 ?
- Yes. The data has **substantial variance** along both features (i.e., both axes)

Dimensionality Reduction: A Simple Illustration



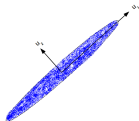
- Now consider a **change of axes** (the co-ordinate system)
- Each example $\mathbf{x}$ has 2 features $\{u_1, u_2\}$
- Consider ignoring the feature u_2 for each example
- Each 2-dimensional example $\mathbf{x}$ now becomes 1-dimensional $\mathbf{x} = \{u_1\}$
- Are we losing much information by throwing away u_2 ?
- No. Most of the data spread is along u_1 (very **little variance** along u_2)

Review:

Principal Component Analysis

Principal Component Analysis (PCA)

- Based on identifying the **Principal Components** in the data
- Principal Components (PC): **Directions of high variance** in the data
- Roughly speaking, PCA does a **change of axes** that represent the data



- First PC: Direction of the **highest variance**
- Second PC: Direction of next highest variability (**orthogonal** to the first PC)
- Subsequent PCs: Other directions of highest variability (in decreasing order)
- Note: All principal components are orthogonal to each other
- PCA: Take top K PC's and project the data along those

PCA: Finding the Principal Components

- Given: N examples $\mathbf{x}_1, \dots, \mathbf{x}_N$, each example $\mathbf{x}_n \in \mathbb{R}^D$
- Goal: Project the data from D dimensions to K dimensions ($K < D$)
- Want projection directions s.t. the projected data has **maximum variance**
 - Note: This is equivalent to **minimizing the reconstruction error**, i.e., error in reconstructing the original data from its projections
- Let $\mathbf{w}_1, \dots, \mathbf{w}_D$ be the principal components, assumed to be:
 - Orthogonal: $\mathbf{w}_i^\top \mathbf{w}_j = 0$ if $i \neq j$, Orthonormal: $\mathbf{w}_i^\top \mathbf{w}_i = 1$
- Each principal component is a vector of size $D \times 1$
- We want only the first K principal components

PCA: Finding the Principal Components

- Projection of a data point $\mathbf{x}_n$ along $\mathbf{w}_1$: $\mathbf{w}_1^\top \mathbf{x}_n$
- Projection of the mean $\bar{\mathbf{x}}$ along $\mathbf{w}_1$: $\mathbf{w}_1^\top \bar{\mathbf{x}}$ (where $\bar{\mathbf{x}} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n$)
- **Variance** of the projected data (along projection direction $\mathbf{u}_1$):

$$\frac{1}{N} \sum_{n=1}^N \left\{ \mathbf{w}_1^\top \mathbf{x}_n - \mathbf{w}_1^\top \bar{\mathbf{x}} \right\}^2 = \mathbf{w}_1^\top \mathbf{S} \mathbf{w}_1$$

where $\mathbf{S}$ is the data **covariance matrix** defined as

$$\mathbf{S} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^\top$$

- Want to have $\mathbf{w}_1$ that **maximizes the projected data variance** $\mathbf{w}_1^\top \mathbf{S} \mathbf{w}_1$
 - Subject to the constraint: $\mathbf{w}_1^\top \mathbf{w}_1 = 1$
 - We will introduce a Lagrange multiplier λ_1 for this constraint

PCA: Finding the Principal Components

- Objective function: $\mathbf{w}_1^\top \mathbf{S} \mathbf{w}_1 + \lambda_1(1 - \mathbf{w}_1^\top \mathbf{w}_1)$
- Taking derivative w.r.t. $\mathbf{w}_1$ and setting it to zero gives:

$$\mathbf{S} \mathbf{w}_1 = \lambda_1 \mathbf{w}_1$$

- This is the eigenvalue equation
 - $\mathbf{w}_1$ must be *an* eigenvector of $\mathbf{S}$ (and λ_1 the corresponding eigenvalue)
- But there are multiple eigenvectors of $\mathbf{S}$. Which one is $\mathbf{w}_1$?
- Consider $\mathbf{w}_1^\top \mathbf{S} \mathbf{w}_1 = \mathbf{w}_1^\top \lambda_1 \mathbf{w}_1 = \lambda_1$ (using $\mathbf{w}_1^\top \mathbf{w}_1 = 1$)
- We know that the projected data variance $\mathbf{w}_1^\top \mathbf{S} \mathbf{w}_1 = \lambda_1$ is maximum
 - Thus λ_1 should be the **largest eigenvalue**
 - Thus $\mathbf{w}_1$ is the **first (top) eigenvector** of $\mathbf{S}$ (with eigenvalue λ_1)
 $\Rightarrow$ the **first principal component** (direction of highest variance in the data)
- Subsequent PC's are given by the subsequent eigenvectors of $\mathbf{S}$

PCA: The Algorithm

- Compute the mean of the data

$$\bar{\mathbf{x}} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n$$

- Compute the sample covariance matrix (using the mean subtracted data)

$$\mathbf{S} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^\top$$

- Do the **eigenvalue decomposition** of the $D \times D$ matrix $\mathbf{S}$
- Take the **top K eigenvectors** (corresponding to the top K eigenvalues)
- Call these $\mathbf{w}_1, \dots, \mathbf{w}_K$ (s.t. $\lambda_1 \geq \lambda_2 \geq \dots \lambda_{K-1} \geq \lambda_K$)
- $\mathbf{W} = [\mathbf{w}_1 \ \mathbf{w}_2 \ \dots \ \mathbf{w}_K]$ is the projection matrix of size $D \times K$
- Projection of each example $\mathbf{x}_n$ is computed as $\mathbf{z}_n = \mathbf{W}^\top \mathbf{x}_n$
 - $\mathbf{z}_n$ is a $K \times 1$ vector (also called the **embedding** of $\mathbf{x}_n$)

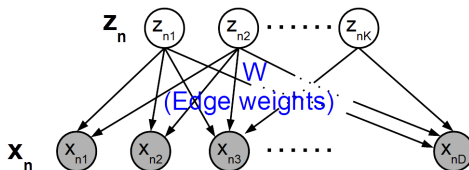
Now on to Probabilistic PCA

Probabilistic PCA (PPCA)

- Assume the following generative model for each observation $\mathbf{x}_n$

$$\mathbf{x}_n = \mathbf{W}\mathbf{z}_n + \epsilon_n$$

- Note: We'll assume data to be centered, otherwise $\mathbf{x}_n = \boldsymbol{\mu} + \mathbf{W}\mathbf{z}_n + \epsilon_n$
- Think of it as low dimensional $\mathbf{z}_n \in \mathbb{R}^K$ “generating” a higher-dimensional $\mathbf{x}_n \in \mathbb{R}^D$ via a mapping matrix $\mathbf{W} \in \mathbb{R}^{D \times K}$, plus some noise $\epsilon_n \sim \mathcal{N}(0, \sigma^2 \mathbf{I}_D)$



- Intuitively, this generative model is “inverse” of what the traditional PCA does. Here we assume a latent low-dim $\mathbf{z}_n$ that “generates” the high-dim $\mathbf{x}_n$ via the mapping $\mathbf{W}$ (plus adding some noise)
- A **directed graphical model** linking $\mathbf{z}_n$ and $\mathbf{x}_n$ via “edge weights” $\mathbf{W}$

Interpreting Probabilistic PCA

- Can also write $\mathbf{x}_n = \mathbf{W}\mathbf{z}_n + \epsilon_n$ as each example $\mathbf{x}_n$ being a linear comb. of columns of $\mathbf{W} = [\mathbf{w}_1, \dots, \mathbf{w}_K]$, plus some example-specific random noise ϵ_n

$$\mathbf{x}_n = \sum_{k=1}^K \mathbf{w}_k z_{nk} + \epsilon_n$$

- The K columns of $\mathbf{W}$ (each $\mathbb{R}^D$) are like “prototype vectors” shared by all examples. Each $\mathbf{x}_n$ is a **linear combination** of these vectors (the combination coefficients are given by $\mathbf{z}_n \in \mathbb{R}^K$ which is basically the low-dim rep. of $\mathbf{x}_n$).
- Some examples:
 - In case of images, columns of $\mathbf{W}$ would correspond to “basis images”
 - In case of text documents, columns of $\mathbf{W}$ (with non-negativity imposed on it) would correspond to “topics” in the corpus

Probabilistic PCA

- Since noise $\epsilon_n \sim \mathcal{N}(0, \sigma^2)$ is Gaussian, the conditional distrib. of $\mathbf{x}_n$

$$p(\mathbf{x}_n | \mathbf{z}_n, \mathbf{W}, \sigma^2) = \mathcal{N}(\mathbf{W}\mathbf{z}_n, \sigma^2 \mathbf{I}_D)$$

- Given a set of observations $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$, the goal is to learn $\mathbf{W}$ and the low-dim. representation of data, i.e., $\mathbf{Z} = \{\mathbf{z}_1, \dots, \mathbf{z}_N\}$

- Assume a Gaussian prior on the low-dimensional latent representation, i.e.,

$$p(\mathbf{z}_n) = \mathcal{N}(0, \mathbf{I}_K)$$

- Using the equation for marginal of Gaussians (lecture-2 and PRML 2.115), the marginal distribution of $\mathbf{x}_n$ (after integrating out latent variables $\mathbf{z}_n$)

$$p(\mathbf{x}_n) \text{ or } p(\mathbf{x}_n | \mathbf{W}, \sigma^2) = \mathcal{N}(\mathbf{0}, \mathbf{W}\mathbf{W}^\top + \sigma^2 \mathbf{I}_D)$$

- Note: Cov. matrix of $p(\mathbf{x}_n)$ now has DK parameters instead of $D(D-1)/2$
- Thus PPCA also allows a more parsimonious parameterization of $p(\mathbf{x}_n)$

Probabilistic PCA

- Consider the marginal likelihood

$$p(\mathbf{x}_n | \mathbf{W}, \sigma^2) = \mathcal{N}(\mathbf{0}, \mathbf{W}\mathbf{W}^\top + \sigma^2 \mathbf{I}_D)$$

- Can do MLE on this, or use EM with latent variables $\mathbf{z}_n$ (is simpler)
- To do EM, we will need the posterior over latent vars. $\mathbf{z}_n$ in the E step
- The posterior over $\mathbf{z}_n$ (using result from lecture-2 and PRML 2.116)

$$p(\mathbf{z}_n | \mathbf{x}_n, \mathbf{W}) = \mathcal{N}(\mathbf{M}^{-1} \mathbf{W}^\top \mathbf{x}_n, \sigma^2 \mathbf{M}^{-1})$$

where $\mathbf{M} = \mathbf{W}^\top \mathbf{W} + \sigma^2 \mathbf{I}_K$ (a $K \times K$ matrix)

EM for Probabilistic PCA

- Observed data: $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$, latent variable: $\mathbf{Z} = \{\mathbf{z}_1, \dots, \mathbf{z}_N\}$
- Parameters: $\mathbf{W}, \sigma^2$
- The complete data log -likelihood

$$\begin{aligned}\log p(\mathbf{X}, \mathbf{Z} | \mathbf{W}, \sigma^2) &= \log \prod_{n=1}^N p(\mathbf{x}_n, \mathbf{z}_n | \mathbf{W}, \sigma^2) = \log \prod_{n=1}^N p(\mathbf{x}_n | \mathbf{z}_n, \mathbf{W}, \sigma^2) p(\mathbf{z}_n) \\ &= \sum_{n=1}^N \{ \log p(\mathbf{x}_n | \mathbf{z}_n, \mathbf{W}, \sigma^2) + \log p(\mathbf{z}_n) \}\end{aligned}$$

- Plugging in, simplifying and ignoring the constants, we get

$$\log p(\mathbf{X}, \mathbf{Z} | \mathbf{W}, \sigma^2) = \sum_{n=1}^N \left\{ \frac{D}{2} \log \sigma^2 + \frac{1}{2\sigma^2} \|\mathbf{x}_n\|^2 - \frac{1}{\sigma^2} \mathbf{z}_n^\top \mathbf{W}^\top \mathbf{x}_n + \frac{1}{2\sigma^2} \text{tr}(\mathbf{z}_n \mathbf{z}_n^\top \mathbf{W}^\top \mathbf{W}) + \frac{1}{2} \text{tr}(\mathbf{z}_n \mathbf{z}_n^\top) \right\}$$

- The expected complete data log-likelihood $\mathbb{E}[\log p(\mathbf{X}, \mathbf{Z} | \mathbf{W}, \sigma^2)]$

$$= \sum_{n=1}^N \left\{ \frac{D}{2} \log \sigma^2 + \frac{1}{2\sigma^2} \|\mathbf{x}_n\|^2 - \frac{1}{\sigma^2} \mathbb{E}[\mathbf{z}_n]^\top \mathbf{W}^\top \mathbf{x}_n + \frac{1}{2\sigma^2} \text{tr}(\mathbb{E}[\mathbf{z}_n \mathbf{z}_n^\top] \mathbf{W}^\top \mathbf{W}) + \frac{1}{2} \text{tr}(\mathbb{E}[\mathbf{z}_n \mathbf{z}_n^\top]) \right\}$$

EM for Probabilistic PCA

- The expected complete data log-likelihood

$$\sum_{n=1}^N \left\{ \frac{D}{2} \log \sigma^2 + \frac{1}{2\sigma^2} \|x_n\|^2 - \frac{1}{\sigma^2} \mathbb{E}[\mathbf{z}_n]^\top \mathbf{W}^\top x_n + \frac{1}{2\sigma^2} \text{tr}(\mathbb{E}[\mathbf{z}_n \mathbf{z}_n^\top] \mathbf{W}^\top \mathbf{W}) + \frac{1}{2} \text{tr}(\mathbb{E}[\mathbf{z}_n \mathbf{z}_n^\top]) \right\}$$

- We need two terms: $\mathbb{E}[\mathbf{z}_n]$ and $\mathbb{E}[\mathbf{z}_n \mathbf{z}_n^\top]$
- We have already seen that

$$p(\mathbf{z}_n | x_n, \mathbf{W}) = \mathcal{N}(\mathbf{M}^{-1} \mathbf{W}^\top x_n, \sigma^2 \mathbf{M}^{-1}) \quad \text{where } \mathbf{M} = \mathbf{W}^\top \mathbf{W} + \sigma^2 \mathbf{I}_K$$

- From posterior $p(\mathbf{z}_n | x_n, \mathbf{W})$, we can easily compute the required expectations

$$\begin{aligned} \mathbb{E}[\mathbf{z}_n] &= \mathbf{M}^{-1} \mathbf{W}^\top x_n \\ \mathbb{E}[\mathbf{z}_n \mathbf{z}_n^\top] &= \mathbb{E}[\mathbf{z}_n] \mathbb{E}[\mathbf{z}_n]^\top + \text{cov}(\mathbf{z}_n) = \mathbb{E}[\mathbf{z}_n] \mathbb{E}[\mathbf{z}_n]^\top + \sigma^2 \mathbf{M}^{-1} \end{aligned}$$

- Taking the derivative of $\mathbb{E}[\log p(\mathbf{X}, \mathbf{Z} | \mathbf{W}, \sigma^2)]$ w.r.t. $\mathbf{W}$ and setting to zero

$$\mathbf{W} = \left[\sum_{n=1}^N x_n \mathbb{E}[\mathbf{z}_n]^\top \right] \left[\sum_{n=1}^N \mathbb{E}[\mathbf{z}_n \mathbf{z}_n^\top] \right]^{-1} = \left[\sum_{n=1}^N x_n \mathbb{E}[\mathbf{z}_n]^\top \right] \left[\sum_{n=1}^N \mathbb{E}[\mathbf{z}_n] \mathbb{E}[\mathbf{z}_n]^\top + \sigma^2 \mathbf{M}^{-1} \right]^{-1}$$

The Full EM Algorithm

- Initialize $\mathbf{W}$ and σ^2
- **E step:** Compute the exp. complete data log-lik. using current $\mathbf{W}$ and σ^2

$$\sum_{n=1}^N \left\{ \frac{D}{2} \log \sigma^2 + \frac{1}{2\sigma^2} \|x_n\|^2 - \frac{1}{\sigma^2} \mathbb{E}[\mathbf{z}_n]^\top \mathbf{W}^\top x_n + \frac{1}{2\sigma^2} \text{tr}(\mathbb{E}[\mathbf{z}_n \mathbf{z}_n^\top] \mathbf{W}^\top \mathbf{W}) + \frac{1}{2} \text{tr}(\mathbb{E}[\mathbf{z}_n \mathbf{z}_n^\top]) \right\}$$

where

$$\begin{aligned} \mathbb{E}[\mathbf{z}_n] &= (\mathbf{W}^\top \mathbf{W} + \sigma^2 \mathbf{I}_K)^{-1} \mathbf{W}^\top x_n = \mathbf{M}^{-1} \mathbf{W}^\top x_n \\ \mathbb{E}[\mathbf{z}_n \mathbf{z}_n^\top] &= \text{cov}(\mathbf{z}_n) + \mathbb{E}[\mathbf{z}_n] \mathbb{E}[\mathbf{z}_n]^\top = \mathbb{E}[\mathbf{z}_n] \mathbb{E}[\mathbf{z}_n]^\top + \sigma^2 \mathbf{M}^{-1} \end{aligned}$$

- **M step:** Re-estimate $\mathbf{W}$ and σ^2 (taking derivatives w.r.t. $\mathbf{W}$ and σ^2 , respectively)

$$\begin{aligned} \mathbf{W}_{new} &= \left[\sum_{n=1}^N x_n \mathbb{E}[\mathbf{z}_n]^\top \right] \left[\sum_{n=1}^N \mathbb{E}[\mathbf{z}_n \mathbf{z}_n^\top] \right]^{-1} = \left[\sum_{n=1}^N x_n \mathbb{E}[\mathbf{z}_n]^\top \right] \left[\sum_{n=1}^N \mathbb{E}[\mathbf{z}_n] \mathbb{E}[\mathbf{z}_n]^\top + \sigma^2 \mathbf{M}^{-1} \right]^{-1} \\ \sigma_{new}^2 &= \frac{1}{ND} \sum_{n=1}^N \left\{ \|x_n\|^2 - 2 \mathbb{E}[\mathbf{z}_n]^\top \mathbf{W}_{new}^\top x_n + \text{tr} \left(\mathbb{E}[\mathbf{z}_n \mathbf{z}_n^\top] \mathbf{W}_{new}^\top \mathbf{W}_{new} \right) \right\} \end{aligned}$$

- Set $\mathbf{W} = \mathbf{W}_{new}$ and $\sigma^2 = \sigma_{new}^2$
- If not converged, go back to E step.

EM for PPCA to a PCA-like Algorithm

- Let's see what happens if the noise variance σ^2 goes to 0
- Let's first look at the E step

$$\mathbb{E}[\mathbf{z}_n] = (\mathbf{W}^\top \mathbf{W} + \sigma^2 \mathbf{I}_K)^{-1} \mathbf{W}^\top \mathbf{x}_n = (\mathbf{W}^\top \mathbf{W})^{-1} \mathbf{W}^\top \mathbf{x}_n$$

No need to compute $\mathbb{E}[\mathbf{z}_n \mathbf{z}_n^\top]$ since it will simply be equal to $\mathbb{E}[\mathbf{z}_n] \mathbb{E}[\mathbf{z}_n]^\top$

- Thus, in this case, E step computes an **orthogonal projection** of $\mathbf{x}_n$
- Let's now look at the M step

$$\mathbf{W}_{new} = \left[\sum_{n=1}^N \mathbf{x}_n \mathbb{E}[\mathbf{z}_n]^\top \right] \left[\sum_{n=1}^N \mathbb{E}[\mathbf{z}_n] \mathbb{E}[\mathbf{z}_n]^\top \right]^{-1} = \mathbf{X}^\top \mathbf{\Omega} (\mathbf{\Omega}^\top \mathbf{\Omega})^{-1}$$

where $\mathbf{\Omega} = \mathbb{E}[\mathbf{Z}]$ is an $N \times K$ matrix with row n equal to $\mathbb{E}[\mathbf{z}_n]$

- Thus, in this case, M step finds $\mathbf{W}$ that **minimizes the reconstruction error**

$$\mathbf{W}_{new} = \arg \min_{\mathbf{W}} \|\mathbf{X} - \mathbb{E}[\mathbf{Z}] \mathbf{W}\|^2 = \arg \min_{\mathbf{W}} \|\mathbf{X} - \mathbf{\Omega} \mathbf{W}\|^2$$

Benefits of PPCA over PCA

- Can handle missing data (can treat it as latent variable in E step)
- Doesn't require computing the $D \times D$ cov. matrix of data and doing expensive eigen-decomposition. When K is small (i.e., we only want few eigen vectors), this is especially nice because only inverting $K \times K$ is required
- Easy to “plug-in” PPCA as part of more complex problems, e.g., mixtures of PPCA models for doing nonlinear dimensionality reduction, or subspace clustering (i.e., clustering when data in each cluster lives on a lower dimensional subspace).
- Possible to give it a fully Bayesian treatment (which has many benefits such as inferring K)

Identifiability

- Note that $p(\mathbf{x}_n) = \mathcal{N}(\mathbf{0}, \mathbf{W}\mathbf{W}^\top + \sigma^2\mathbf{I}_D)$
- If we replace $\mathbf{W}$ by $\tilde{\mathbf{W}} = \mathbf{W}\mathbf{R}$ for some orthogonal rotation matrix $\mathbf{R}$ then

$$\begin{aligned} p(\mathbf{x}_n) &= \mathcal{N}(\mathbf{0}, \tilde{\mathbf{W}}\tilde{\mathbf{W}}^\top + \sigma^2\mathbf{I}_D) \\ &= \mathcal{N}(\mathbf{0}, \mathbf{W}\mathbf{R}\mathbf{R}^\top\mathbf{W}^\top + \sigma^2\mathbf{I}_D) \\ &= \mathcal{N}(\mathbf{0}, \mathbf{W}\mathbf{W}^\top + \sigma^2\mathbf{I}_D) \end{aligned}$$

- Thus PPCA doesn't give a unique solution (for every $\mathbf{W}$, there is another $\tilde{\mathbf{W}} = \mathbf{W}\mathbf{R}$ that gives the same solution)
- Thus the PPCA model is not uniquely identifiable
- Usually this is not a problem, unless we want to interpret $\mathbf{W}$
- To ensure identifiability, we can impose certain structure on $\mathbf{W}$, e.g., constrain it to be a lower-triangular or sparse matrix

Factor Analysis

- Similar to PPCA except that the Gaussian conditional distribution $p(\mathbf{x}_n|\mathbf{z}_n)$ has diagonal instead of spherical covariance

$$\mathbf{x}_n \sim \mathcal{N}(\mathbf{W}\mathbf{z}_n, \Psi)$$

where Ψ is a diagonal matrix

- In Factor Analysis, the projection matrix $\mathbf{W}$ is also called the **Factor Loading Matrix** and $\mathbf{z}_n$ is called the **factor scores** for example n
- EM for Factor Analysis is same as that for PPCA except
 - The required expectations in the E step :

$$\begin{aligned}\mathbb{E}[\mathbf{z}_n] &= \mathbf{G}^{-1}\mathbf{W}^\top\Psi^{-1}\mathbf{x}_n \\ \mathbb{E}[\mathbf{z}_n\mathbf{z}_n^\top] &= \mathbb{E}[\mathbf{z}_n]\mathbb{E}[\mathbf{z}_n]^\top + \mathbf{G}\end{aligned}$$

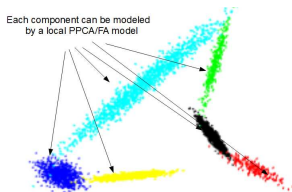
where $\mathbf{G} = (\mathbf{W}^\top\Psi^{-1}\mathbf{W} + \mathbf{I}_K)^{-1}$

- In the M step, in addition to $\mathbf{W}_{new}$, we also need to estimate Ψ

$$\Psi_{new} = \text{diag} \left\{ \mathbf{s} - \mathbf{W}_{new} \frac{1}{N} \sum_{n=1}^N \mathbb{E}[\mathbf{z}_n] \mathbf{x}_n^\top \right\}$$

Mixture of PPCAs/Mixture of Factor Analyzers

- PPCA and FA learn a linear projection of the data (i.e., are linear dimensionality reduction methods)
- Can use mixture of PPCAs or mixture of FAs to learn nonlinear projections (i.e., nonlinear dimensionality reduction)



- Similar to mixture of Gaussians, except that now each Gaussian is replaced by a PPCA or FA model
- Note: Unlike PPCA/FA, can't have $\mu = 0$: Each PPCA/FA based mixture component k will have a nonzero mean μ_k and projection matrix $\mathbf{W}_k$
- We will later look at another nonlinear dim. red. model - Gaussian Process Latent Variable Models (GPLVM)

For details, check out "Mixtures of Probabilistic Principal Component Analysers" by Tipping and Bishop, and "The EM Algorithm for Mixtures of Factor Analyzers" by Ghahramani and Hinton